amazon

BRAZIL

Amazon Brazil

E-Commerce Order Analytics

A SQL Case Study on Customers, Sellers, Products & Fulfillment

14	17	6
Business Questions	SQL Queries	Core Tables

Prepared by: Vinay Mehra | Role: Data Analyst

01 Project Overview & Business Context

The Business Problem

Amazon Brazil is scaling rapidly across Latin America's largest e-commerce market. The operations and strategy team needs a clear, data-driven picture of platform health: which customers drive revenue, which sellers deliver reliably, which categories are growing, and where fulfillment breaks down.

I am analyzing Amazon Brazil's order database spanning customers, sellers, products, orders, payments, and reviews with the goal of writing SQL queries that convert raw transactional data into decisions the business can act on.

This deck walks through 14 real business questions, the SQL logic used to answer each one, and space to attach the resulting output for review.



DATASET

amazon.sql

Data Dictionary: amazon_data_dictionary.txt

- customers
- orders
- order_items
- sellers
- products
- order_payments
- order_reviews

Analytical Themes Covered

- Fulfillment & logistics performance
- Revenue trends by category & time
- Customer spend & loyalty segmentation
- Payment method economics
- Seller performance benchmarking
- Review sentiment & satisfaction by state

Agenda - 14 Business Questions



1. Orders fulfilled by seller state
2. Cumulative revenue by category over time
3. Most-used payment method & avg order value
4. Highest-spending customer
5. Average review score by category
6. Orders by customer, by state
7. Sellers who never fulfilled an order
8. Top 5 categories by revenue
9. Median delivery time
10. Products never ordered
11. Sellers above average order volume
12. States with highest avg review score
13. Customers who never left a review
14. Peak order month

Q1 Orders Fulfilled by Seller State

Regional fulfillment center investment planning

```sql
-- Q1. Find the total number of orders fulfilled by each seller state.
select s.seller_state,
       count(distinct oi.order_id) as no_of_orders
from order_items oi
join sellers s
on oi.seller_id = s.seller_id
group by s.seller_state
order by no_of_orders desc;
```

Business Context

Amazon Brazil's logistics team is planning its next round of regional fulfillment center investments. Before committing budget, they need to know which states are generating the most order volume on the seller side, so infrastructure goes where the actual demand is.

Expected insight: a ranked list of seller states by delivered order count, highlighting where to prioritize warehouse and last-mile capacity.

Outcome: São Paulo (SP) overwhelmingly dominates order fulfillment, with 70,188 orders, far ahead of Minas Gerais (MG) with **7,930** and **Paraná (PR)** with **7,673.**

SP alone accounts for **~71%** of all fulfilled orders in the dataset. The next major states are MG, PR, and RJ, but their volumes are significantly lower. Order fulfillment is therefore highly concentrated in São Paulo, indicating that SP is the primary seller hub.

RESULT

seller_state	no_of_orders
SP	70188
MG	7930
PR	7673
RJ	4353
SC	3667
RS	1989
DF	824
BA	569
GO	463
PE	406
MA	392
ES	318
MT	137
CE	91
RN	51
MS	49
PB	36
RO	14
PI	12
SE	9
PA	8
AM	3
AC	1

Q2 Cumulative Revenue by Category Over Time

Flash sale & push-notification timing

Business Context

The growth team wants to see how revenue builds up within each category across the order timeline. This running total helps them spot which categories hit revenue targets early in the month versus which pick up only toward the end useful for planning flash sales and push notifications.

Outcome: Cumulative revenue tracks how each product category's revenue builds over time as new orders are received. Here, **Agro Industries** revenue is increasing from **₹43.98** to **₹202.85** across the displayed dates.

Th helps **identify how quickly** each product **category generates revenue over time**, enabling comparison of category performance and identification of categories with stronger revenue momentum.

RESULT

product_category_name	purchase_date	total_revenue	cumulative_revenue
agro_industria_e_come...	2017-01-23	43.98	43.98
agro_industria_e_come...	2017-01-31	21.99	65.97
agro_industria_e_come...	2017-02-05	21.99	87.96
agro_industria_e_come...	2017-02-08	21.99	109.94999999999999
agro_industria_e_come...	2017-02-12	92.9	202.85

SQL QUERY

```sql
-- Q2. For each product category, calculate the cumulative revenue generated as orders come in over time.
with category_revenue as
(
select p.product_category_name,
    date(o.order_purchase_timestamp) as purchase_date,
    sum(oi.price) as total_revenue
from products p
join order_items oi
on p.product_id = oi.product_id
join orders o
on oi.order_id = o.order_id
group by
    p.product_category_name,
    date(o.order_purchase_timestamp)
)
 select cr.product_category_name,
 purchase_date,
 total_revenue,
 sum(cr.total_revenue) over (
                        partition by cr.product_category_name
                        order by purchase_date asc
                        ) as cumulative_revenue
 from category_revenue cr;
```

Q3 Payment Method Usage & Avg Order Value

Payment provider contract renegotiation

Business Context

The finance and partnerships team is renegotiating contracts with payment providers.

Before those talks, they need to know which payment method among credit card, boleto, voucher, or debit card dominates the platform, and whether high-value orders cluster around a specific payment type.

Outcome: Credit cards are the most preferred payment method, accounting for **75,991 orders**, significantly higher than other payment types. They also have the highest average order value of **₹166.81**.

Boleto follows with **19,614** orders and an average order value of **₹144.91**, while debit cards and vouchers have much lower usage.

SQL QUERY

```sql
-- Q3. Which payment method do customers use the most, and what is the average order value for each payment type?
with order_value_per_order as
(
select oi.order_id,
    sum(oi.price + oi.freight_value) order_value
from order_items oi
group by oi.order_id
)
select op.payment_type, count(distinct op.order_id) as total_orders, avg(ovpo.order_value) average_order_value
from order_value_per_order ovpo
join order_payments op
on ovpo.order_id = op.order_id
group by op.payment_type
order by average_order_value desc;
```

RESULT

payment_type	total_orders	average_order_value
credit_card	75991	166.80976008809108
boleto	19614	144.9087534414187
debit_card	1521	141.46303547963203
voucher	3766	139.14958170861337

Q3 + Payment Method - Simplified Using order_payments Only

Value-added: same business question, leaner query

Alternative Calculation

This alternative skips the order_items join entirely and uses **payment_value** directly from order_payments, dividing total payment value by distinct order count per payment_type. It's simpler and faster when **freight-level precision isn't required**, and results are sorted by order volume rather than value.

SQL QUERY (ALTERNATIVE APPROACH)

```sql
SELECT
    payment_type,
    COUNT(DISTINCT order_id) AS no_of_orders,
    SUM(payment_value) / COUNT(DISTINCT order_id) AS average_order_value
FROM order_payments
GROUP BY payment_type
ORDER BY no_of_orders DESC;
```

RESULT

payment_type	no_of_orders	average_order_value
credit_card	76505	163.9380980328024
boleto	19784	145.03443540234508
voucher	3866	98.1471469218833
debit_card	1528	142.66347513089
not_defined	3	0

Q4 Highest-Spending Customer

Premium loyalty tier — first member onboarding

Business Context

The CRM team is launching Amazon Brazil's first premium loyalty tier. The first step is identifying the single highest-spending customer on the platform, both to personally onboard them into the program and to understand what a top-tier customer profile looks like.

Outcome:

Customer **'0a0a92112bd4c708ca5fde585afaa872',** is the highest-value customer, with total spending of **₹13,664.08** across all orders.

Key takeaway: This customer represents a significant high-value customer segment, highlighting an opportunity to strengthen retention and personalized engagement strategies for customers with similar spending patterns.

SQL QUERY

```sql
-- Q4. Find the customer who has spent the most money across all their orders.
    select c.customer_unique_id,
        sum(op.payment_value) as money_spent
        from customers c
        join orders o
        on c.customer_id = o.customer_id
        join order_payments op
        on o.order_id = op.order_id
        group by c.customer_unique_id
        order by money_spent desc
        limit 1;
```

RESULT

customer_unique_id	money_spent
0a0a92112bd4c708ca5fde585afaa872	13664.08

Q4 +

Handling Ties - All Top Spenders via DENSE_RANK()

Value-added: robust version for tied spend amounts

Alternative Approach

LIMIT 1 silently **drops** a **tie**: if two customers spent the exact same top amount, only one is returned. This alternative uses **DENSE_RANK()** over total spend so **every customer** sharing the #1 spot is **captured**, the correct approach whenever the business asks for the highest-spending customer(s), plural-safe.

SQL QUERY (ALTERNATIVE APPROACH)

```sql
-- But, but, but, If two customers have spent exactly the same amount, LIMIT 1 returns only one of them. So,
-- If the interviewer asks: Find the highest-spending customer(s) then use DENSE_RANK() instead.

    with customer_spending as
    (
    select c.customer_unique_id,
           sum(op.payment_value) as money_spent
    from customers c
    join orders o
    on c.customer_id = o.customer_id
    join order_payments op
    on o.order_id = op.order_id
    group by c.customer_unique_id
    ),
    customer_spending_ranks as
    (
    select cs.customer_unique_id, cs.money_spent, dense_rank() over(order by cs.money_spent desc) as rnk
    from customer_spending cs
    )
    select * from customer_spending_ranks
    where rnk = 1;
```

RESULT

customer_unique_id	money_spent	rnk
0a0a92112bd4c708ca5fde585afaa872	13664.08	1

Q5 Average Review Score by Product Category

Prioritizing CX intervention by category

Business Context

The customer experience team tracks satisfaction at the category level. A low average review score in a category often signals a fulfillment issue, a product quality problem, or a mismatch between listing description and actual product. This query helps prioritize which categories need intervention first.

Outcome:

Customer satisfaction is strongest in the top-rated categories, with all five recording an average review score above 4.4/5:

- CDs, DVDs & Musicals: 4.64/5
- Fashion - Children & Youth: 4.50/5
- General Interest Books: 4.45/5
- Construction Tools: 4.44/5
- Flowers: 4.42/5

Key takeaway: These categories combine high customer satisfaction with consistently strong ratings, making them positive benchmarks for product quality and customer experience across the platform.

SQL QUERY

```sql
-- Q5. Find the average review score for each product category.
select
    p.product_category_name,
    round(avg(rev.review_score),2) as average_review_score
from products p
join order_items oi
    on p.product_id = oi.product_id
join order_reviews rev
    on oi.order_id = rev.order_id
group by p.product_category_name
order by average_review_score desc;
```

RESULT

product_category_name	average_review_score
cds_dvds_musicais	4.64
fashion_roupa_infanto_juvenil	4.50
livros_interesse_geral	4.45
construcao_ferramentas_ferramentas	4.44
flores	4.42

Q6 Orders by Customer, Broken Down by State

State-level marketing campaign planning

Business Context

The regional marketing team is planning state-level campaigns in Brazil.

They want to know how order frequency varies by customer location, so they can target low-frequency states with acquisition offers and reward high-frequency states with loyalty perks.

Outcome:

The results indicate a largely single-order customer base within these states, highlighting potential opportunities to improve repeat purchases and customer retention through targeted engagement and loyalty initiatives.

SQL QUERY

```sql
-- Q6. Find the total number of orders placed by each customer, broken down by the state they live in.
select c.customer_id,
       c.customer_state,
       count(distinct o.order_id) as no_of_orders
from orders o
join customers c
on o.customer_id =c.customer_id
group by c.customer_id, c.customer_state
order by c.customer_state asc;
```

RESULT

91b908a3599b613c9ebc32c50fcc1b3a	AC	1
8ff83690a72dc5589bc39742798a128b	AC	1
8d1d9151c6eb5fb90b19d52e8b337e2f	AC	1
8cd8dcd00b3204df8a1d464ae879c8c5	AL	1
8d776ed4b01be6ade388915380a7c6b9	AL	1

Q7 Sellers Who Never Fulfilled an Order

Quarterly dormant-seller re-engagement

Business Context

The seller growth team runs quarterly re-engagement campaigns targeting dormant sellers. To build that list, they need every seller account that exists in the system but has zero fulfilled orders, these accounts need a nudge, or review for removal.

Outcome:

No sellers were identified as having registered on the platform without fulfilling an order. The analysis returned no records, indicating that **every registered seller** in the dataset has **fulfilled** at **least one order.**

Key takeaway: The **seller base** shows **100% order participation**, with no evidence of inactive sellers who registered but never completed a sale.

SQL QUERY

```sql
-- Q7. Identify sellers who registered on the platform but have never fulfilled a single order.
select s.seller_id,
       oi.order_id,
       oi.order_item_id
from sellers s
left join order_items oi
on oi.seller_id = s.seller_id
where oi.order_id is null;
```

RESULT

seller_id	order_id	order_item_id

Q8 Top 5 Product Categories by Total Revenue

Homepage placement & promo budget decisions

Business Context

Before every major sales event, Black Friday Brazil, Dia das Mães, Cyber Monday etc., the merchandising team decides which categories get homepage real estate and promotional budget.

Total revenue by category is the primary input for that decision.

Outcome:

Health & Beauty (beleza_saude) leads with **R$ 1.26M**, followed by Watches & Gifts (**R$ 1.21M**) and Bed, Bath & Table (**R$ 1.04M**).

Key Takeaway: Prioritize leading categories for promotions and inventory planning while exploring growth opportunities in other high-performing categories.

Revenue reflects item prices, excluding freight.

SQL QUERY

```sql
-- Q8. Find the top 5 product categories by total revenue.
-- Based on Price
SELECT
    p.product_category_name,
    ROUND(SUM(oi.price), 2) AS total_revenue
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY
    p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

RESULT

product_category_name	total_revenue
beleza_saude	1258681.34
relogios_presentes	1205005.68
cama_mesa_banho	1036988.68
esporte_lazer	988048.97
informatica_acessorios	911954.32

Q9 Median Delivery Time (Days)

True typical wait time for customers

Business Context

The logistics director wants the true middle ground of how long customers wait for their orders. Median is preferred over average here because a handful of extremely delayed orders due to remote delivery locations in Brazil's interior would skew the average and make performance look worse than it is for most customers.

Outcome:

Median is useful here because it is **less affected by** unusually delayed orders (**outliers**) than the average.

Here, the Median delivery time is **10 days**.

Overall, customers typically received their orders in around 10 days from order placement to actual delivery. If the company wants **to improve customer experience**, **reducing** this **median delivery time** would indicate an **improvement** in overall delivery efficiency.

SQL QUERY

```sql
-- Q9. Find the median delivery time (in days) between order placement and actual delivery.
with delivery_time as
(
    select
          order_id,
          DATEDIFF(
                order_delivered_customer_date,
                order_purchase_timestamp
          ) as delivery_days
    from orders
    where order_delivered_customer_date is not null
),
ranked as
(
    select
          order_id,
          delivery_days,
          row_number() over(
                order by delivery_days
          ) as rn,
          COUNT(*) over() as total_orders
    from delivery_time
)
select
     avg(delivery_days) as median_delivery_days
from ranked
where rn in
(
   FLOOR((total_orders + 1) / 2),
   CEIL((total_orders + 1) / 2)
);
```

RESULT

median_delivery_days
10.0000

Q10 Products That Have Never Been Ordered

Quarterly dead-listing clean-up

Business Context

The logistics director wants the true middle ground of how long customers wait for their orders.

Outcome:

Median is **preferred** over average here because a handful of **extremely delayed orders**, due to **remote** delivery **locations** in Brazil's interior, would **skew** the **average** and make performance look worse than it is for most customers.

Here, the Median delivery time is **10 days**.

Overall, customers typically received their orders in around 10 days from order placement to actual delivery. If the company wants **to improve customer experience**, **reducing** this **median delivery time** would indicate an **improvement** in overall delivery efficiency.

SQL QUERY

```sql
-- Q10. Find all products that have never been ordered.
select p.product_id, oi.order_id
from products p
left join order_items oi
on p.product_id = oi.product_id
where oi.order_id is null;
```

RESULT

product_id	order_id

Q11 Sellers Above Average Order Volume

"Top Seller" badge program eligibility

Business Context

The seller success team is building an invite list for Amazon Brazil's "Top Seller" badge program. Only sellers who perform above the platform average qualify. This query identifies exactly who clears that bar and gives the team a baseline for what above-average performance actually means numerically.

Outcome:

All the **sellers shown** have fulfilled **more than** the platform-wide **average** of **32.31 orders**. This indicates that these sellers are above-average contributors in terms of order fulfillment volume.

For example, the **seller with 200 orders** has fulfilled roughly **6.2 times** the **average seller's order volume**, making it a particularly **high-volume seller**.

Key insight: Sellers exceeding the average order volume can be considered high-performing or high-volume sellers and may be important for the platform in terms of sales contribution and seller management.

SQL QUERY

```sql
-- Q11. Find sellers who have fulfilled more orders than the average seller on the platform.
with seller_orders as
(
select s.seller_id,
       count(distinct oi.order_id) as total_no_of_orders
from sellers s
left join order_items oi
on s.seller_id = oi.seller_id
group by s.seller_id
),
average_orders as
(
select so.seller_id,
       so.total_no_of_orders,
       avg(so.total_no_of_orders) over() as average_no_of_orders
from seller_orders so
)
select ao.seller_id,
       ao.total_no_of_orders,
       ao.average_no_of_orders
from average_orders ao
where ao.total_no_of_orders > ao.average_no_of_orders;
```

RESULT

seller_id	total_no_of_orders	average_no_of_orders
001cca7ae9ae17fb1caed9dfb1094831	200	32.3134
002100f778ceb8431b7a1020ff7ab48f	51	32.3134
004c9cd9d87a3c30c522c48c4fc07416	158	32.3134
00ee68308b45bc5e2660cd833c3f81cc	135	32.3134
00fc707aaaad2d31347cf883cd2dfe10	103	32.3134

Q11 + Counting Qualifying Sellers

Value-added: single KPI number for the badge program

Value-Added Alternative

Building directly on the above-average seller CTE, this variant wraps it in one more layer and returns a single COUNT(*), the exact number of sellers who qualify for the Top Seller badge. Useful as a headline KPI for leadership decks rather than a full seller-by-seller list.

SQL QUERY (ALTERNATIVE APPROACH)

```sql
with seller_orders as
(
select s.seller_id,
       count(distinct oi.order_id) as total_no_of_orders
from sellers s
left join order_items oi
on s.seller_id = oi.seller_id
group by s.seller_id
),
average_orders as
(
select so.seller_id,
       so.total_no_of_orders,
       avg(so.total_no_of_orders) over() as average_no_of_orders
from seller_orders so
),
above_average_sellers as
(
select ao.seller_id,
       ao.total_no_of_orders,
       ao.average_no_of_orders
from average_orders ao
where ao.total_no_of_orders > ao.average_no_of_orders
)
select COUNT(*) as number_of_above_average_sellers
from above_average_sellers;
```

Outcome:

Only **19.06%** of sellers fulfilled **more orders than** the platform-wide **average** of **32.31 orders**.

This indicates that a **relatively small portion** of sellers are operating **above** the **average** fulfillment volume, while approximately **80.94%** of sellers are at or **below the average**.

This suggests that **order fulfillment** is **concentrated** among a **smaller group of high-volume sellers**, making these sellers particularly important for the platform's overall order activity.

RESULT

number_of_above_average_sellers
590

Q12 States with Highest Avg Review Score

Correlating delivery destination with satisfaction

Business Context

The last-mile delivery experience varies significantly across Brazil. This query helps the CX team correlate delivery destination with satisfaction, so they can push logistics partners to improve in underperforming states.

Outcome:

Amapá (AP) has the **highest** average customer review score at **4.19/5**, indicating a very positive customer experience for orders delivered in the state.

Paraná (PR) and **Amazonas (AM)** follow closely at **4.18**, while **São Paulo (SP)** records an average of **4.17**.

The **difference** between the highest score (4.19) and the fifth-highest (4.13) **is** only **0.06 points.**

Therefore, although AP ranks first, the **performance gap** among the top states is **very small**. However, the **narrow** score **range suggests** that there is **no major difference** in **customer satisfaction among** the **leading states**.

SQL QUERY

```sql
-- Q12. Find which Brazilian states have the highest average customer review score for orders delivered there.
with state_wise_reviews as
(
select c.customer_state,
       round(avg(rev.review_score),2) as average_customer_review_score
from customers c
join orders o
on c.customer_id = o.customer_id
join order_reviews rev
on o.order_id = rev.order_id
group by c.customer_state
)
select swr.customer_state,
       swr.average_customer_review_score,
       dense_rank() over(order by average_customer_review_score desc) as state_rank
from state_wise_reviews swr;
```

RESULT

customer_state	average_customer_review_score	state_rank
AP	4.19	1
PR	4.18	2
AM	4.18	2
SP	4.17	3
MG	4.14	4
RS	4.13	5

Q13 Customers Who Ordered but Never Reviewed

Growing review volume for trust & discoverability

Business Context

The post-purchase engagement team wants to grow the volume of reviews on the platform, More reviews mean better product discoverability and buyer trust.

The first step is identifying customers who buy regularly but never review, so the team can target them with review reminder emails and in-app nudges.

Outcome:

These customers represent a **potential customer-feedback gap.**

They have completed the purchasing journey but have not provided any review, meaning the platform is not receiving feedback from this segment.

This can be important because **reviews help** the platform and sellers **understand customer satisfaction**, **product quality**, **delivery experience**, and **potential service issues.**

SQL QUERY

```sql
-- Q13. Identify customers who have placed orders but never left a review.
select distinct c.customer_id
from customers c
join orders o
on c.customer_id = o.customer_id
left join order_reviews rev
on o.order_id = rev.order_id
where rev.review_id is null;
```

RESULT (List is Non-exhaustive)

customer_id
5f174de9b3c05f164681fb0f3e47f453
8b28d65024bde74b024ae8c9326adf50
4a6f6cd7b3fa8d3f9a370adfdad0fa97
04361b218639fa1eea85ab84aebcab7f
404e67122d00aaf4e82413c0355aca9d

Q13+ Number of Customers Who Ordered but Never Reviewed

Business Context

Identifying the number of customers who have purchased but never provided a review helps the business understand the size of this feedback gap.

Outcome:
768 out of **99,441 customers (0.77%)** placed orders but never left a review.

This represents a relatively **small segment** of the overall customer base. However, these customers still represent a **potential opportunity** to collect additional feedback.

This has **two** important **implications**:

Feedback gap: The platform may be **missing valuable information** about customers' actual experience with products, sellers, and delivery services.

Engagement opportunity: These **customers could** potentially be **encouraged** to leave reviews **through** post-delivery **notifications**, simplified review processes, **or incentives**.

SQL QUERY

```sql
select count(distinct c.customer_id) as no_of_reviewless_customers
from customers c
join orders o
on c.customer_id = o.customer_id
left join order_reviews rev
on o.order_id = rev.order_id
where rev.review_id is null;
```

RESULT (List is Non-exhaustive)

no_of_reviewless_customers
768

Q14 Peak Order Month

Warehouse capacity & seasonal hiring decisions

Business Context

The demand forecasting team needs to know Amazon Brazil's peak order month historically. This drives decisions about how much warehouse capacity to reserve, how many temporary delivery staff to hire, and when to start marketing campaigns for the next peak season.

Outcome:

November 2017 was the platform's **strongest month** in terms of **order volume**, with **7,544 orders.**

Understanding the factors behind this peak can **help** the platform replicate successful **strategies** and prepare **operational capacity** for similar periods of high demand.

The business could **investigate** whether this **spike** was **driven by seasonal shopping**, promotional **campaigns**, or increased **customer activity**.

SQL QUERY

```sql
-- Q14. Find the month with the highest number of orders placed across the entire platform.
SELECT
    DATE_FORMAT(order_purchase_timestamp, '%b %Y') AS order_month,
    COUNT(*) AS total_orders
FROM orders
GROUP BY DATE_FORMAT(order_purchase_timestamp, '%b %Y')
ORDER BY total_orders DESC
LIMIT 1;
```

RESULT

order_month	total_orders
Nov 2017	7544

Key Takeaways

- 14 business questions answered across fulfillment, revenue, customers, sellers & satisfaction
- 17 SQL queries, including robust, tie-safe, and simplified alternative approaches
- Every query maps directly to a real Amazon Brazil operations or growth decision

Thank you

By Vinay Mehra